MA1508E MATLAB Reference

Generated on May 1, 2026

MA1508E — MATLAB Function Reference

A reference for every `.m` file in `Telegram Desktop/MATLAB`. The first section is a quick-lookup table grouped by chapter; the second section gives a worked example for each core function.

All scripts default to **exact symbolic arithmetic** so workings stay in fraction / surd form (course requirement).

1. Quick reference table

Chapter 1 — Linear Systems

Function	Purpose	Call
`solveLinearSystem_RREF`	Solve $A\mathbf{x} = \mathbf{b}$ by RREF: prints augmented matrix, RREF, status, general solution.	`solveLinearSystem_RREF(A,b)`
`classify_linear_system`	Parametric system: detects critical conditions (RREF denominators, consistency rows, $\det A$, left-null space), enumerates each case, and recurses into joint conditions for **up to 4 parameters**. Distinguishes infinite-solution cases by free-parameter count (1 param `s`, 2 params `s,t`, …). Ends with a roll-up summary table of all leaf cases.	`classify_linear_system(M)` or `(A,b)`
`symbolicREF`	Symbolic Gaussian elimination to row-echelon form (no early division by symbolic expressions).	`R = symbolicREF(A)`
`symbolicRREF`	Exact symbolic RREF with case branching for parametric pivots.	`symbolicRREF(A)`
`findEROsequence`	Identify the elementary row operations transforming A into B.	`findEROsequence(A,B)`
`elementaryMatrices`	Build elementary matrices for a sequence of row ops + composite + inverse composite.	`elementaryMatrices(ops, n)`
`parametricCases`	Case-by-case symbolic split when pivots vanish for special parameter values.	`parametricCases(A,{'a',[v1 v2]})`
`solveMatrixEquation`	Solve $AM = B$ for A, optionally augmented with known null-space columns.	`solveMatrixEquation(M,B)`

Chapter 2 — Matrix Algebra & Determinants

Function	Purpose	Call
`cofactor_expansion`	Compute $\det(A)$ by cofactor expansion along a row/column.	`cofactor_expansion(A)`
`cofactorMatrix`	Compute the cofactor matrix (use to build the adjugate).	`C = cofactorMatrix(A)`

Chapter 3 — Vector Spaces, Span, Bases

Function	Purpose	Call
`inSpan`	Decide $\mathbf{v} \in \operatorname{span}(S)$; if yes, return one valid coefficient vector.	`inSpan(v,S)`
`coordinatesInBasis`	Return $[\mathbf{v}]_B$ exactly, given a basis B (columns).	`coordinatesInBasis(v,B)`
`isLinearlyIndependent`	Decide independence (handles zero vector, $k > n$, etc.); on failure prints the dependence relation.	`isLinearlyIndependent(A)`
`compare_spans`	Verbose comparison of $\operatorname{span}(A)$ vs $\operatorname{span}(B)$ (subset / equal / disjoint).	`compare_spans(A,B)`
`extendToBasis`	Extend independent vectors in $\mathbb{R}^n$ to a basis.	`extendToBasis(V)`
`polynomialInterpolation`	Polynomial of degree $\leq n$ through given points; introduces free parameters when underdetermined.	`polynomialInterpolation(xs,ys, deg)`
`vandermondeMatrix`	Vandermonde matrix for given x-values up to a chosen max degree d. Same column convention as `polynomialInterpolation` (highest-power-first).	`V = vandermondeMatrix(xs, deg)`
`changeBasisCoords`	Convert $[\mathbf{w}]_S$ to $[\mathbf{w}]_T$ when S and T are bases of the same subspace.	`changeBasisCoords(coordsS, S, T)`

Chapter 4 — Subspaces of a Matrix

Function	Purpose	Call
`rowColSpace`	Bases for $\operatorname{Row}(A)$ and $\operatorname{Col}(A)$.	`rowColSpace(A)`
`subspaceFromEquations`	Basis for $V = \{\mathbf{x} : C\mathbf{x} = \mathbf{0}\}$ — i.e. $\operatorname{Null}(C)$ — when V is given by linear equations rather than a spanning set.	`subspaceFromEquations(C)`
`find_intersection`	Basis for $U \cap V$ given spanning matrices.	`find_intersection(A,B)`
`left_inverse`	Left inverse $(A^T A)^{-1} A^T$ when A has full column rank.	`L = left_inverse(A)`
`right_inverse`	Right inverse $A^T (AA^T)^{-1}$ when A has full row rank.	`R = right_inverse(A)`

Chapter 5 — Orthogonality, Projection, Least Squares

Function	Purpose	Call
`projectOnto`	Projection $\mathrm{proj}_{\mathbf{u}}(\mathbf{w})$ with full workings: prints $\langle \mathbf{w}, \mathbf{u} \rangle$, $\langle \mathbf{u}, \mathbf{u} \rangle$, the scalar c, and the result.	`projectOnto(w,u)`
`decomposeProjection`	Split $\mathbf{w} = \mathbf{w}_p + \mathbf{w}_n$ where $\mathbf{w}_p \in \mathrm{Col}(V)$, $\mathbf{w}_n \perp \mathrm{Col}(V)$.	`decomposeProjection(w,V)`
`orthogonalComplement`	Basis of $V^{\perp} = \mathrm{Null}(V^T)$.	`orthogonalComplement(V)`
`gramSchmidt`	Gram-Schmidt (symbolic internally): prints every dot product, projection, and norm step in fraction/surd form; final orthonormal set in factored $(1/\sqrt{c})\,\mathbf{z}$ form.	`gramSchmidt(V)`
`orthonormalize_rational`	Gram-Schmidt with exact rational/symbolic output (full surd workings, errors only on linearly dependent columns).	`orthonormalize_rational(V)`
`isOrthogonal`	Pairwise orthogonality check (decimal tolerance).	`isOrthogonal(V)`
`isOrthonormal`	Exact symbolic orthonormality check (unit length **and** pairwise orthogonal).	`isOrthonormal(V)`
`least_squares`	Least-squares solution of $A\mathbf{x} = \mathbf{v}$ via normal equations, with full RREF workings, projection, residual, and error norm. Handles unique and infinite-solution cases.	`[x,p,Wo,dist] = least_squares(A,v)`

Chapter 6 — Eigenanalysis & Markov Chains

Function	Purpose	Call
`charPoly`	$\det(xI - A)$ expanded, factored, with algebraic + geometric multiplicities.	`charPoly(A)`
`eigenAnalysis`	Eigenspace bases + diagonalisation $A = PDP^{-1}$ (orthogonal P if symmetric); when defective, builds Jordan chains and returns Jordan form J with $A = PJP^{-1}$.	`[P,D] = eigenAnalysis(A)`
`orthogonalDiagonalize`	Symmetric A: $A = PDP^T$ with orthogonal P.	`orthogonalDiagonalize(A)`
`calculateSteadyState`	Exact steady-state vector of a stochastic matrix P.	`calculateSteadyState(P)`
`isStochastic`	Column-stochastic check.	`isStochastic(P)`
`isRegularStochastic`	Returns true if some power $P^k > 0$ (regular).	`isRegularStochastic(P)`

Chapter 7 — Linear Differential Equations

Function	Purpose	Call
`wronskian`	$W(\mathbf{x}_1, \ldots, \mathbf{x}_n)$ + independence verdict.	`wronskian(X)`
`solveLinearODE`	Solve $\mathbf{y}' = A\mathbf{y}$ (general or IVP). Walks Jordan blocks manually — output stays **real for complex eigenvalues** (cos/sin form, no `1i` leakage), and each fundamental solution is rescaled to clear rational denominators (e.g. $[\frac{1}{2}, \frac{1}{2}, 1]^T \to [1, 1, 2]^T$). Prints fundamental solutions then general/particular.	`solveLinearODE(A,x0,t0)`
`scalarODEtoSystem`	Companion matrix for a scalar n-th order ODE $c_n y^{(n)} + \cdots + c_0 y = 0$. Returns A such that $Y' = AY$ where $Y = [y; y'; \ldots; y^{(n-1)}]$ — pass directly into `solveLinearODE`.	`A = scalarODEtoSystem([c_n, …, c_0])`
`generalizedEigenvector`	Solve $(A - \lambda I)\mathbf{v}_2 = \mathbf{v}_1$ for repeated eigenvalues.	`generalizedEigenvector(A,lam,v1)`

Beyond core (SVD)

Function	Purpose	Call
`svd_exact`	Symbolic/exact SVD: $A = USV^T$.	`[U,S,V] = svd_exact(A)`
`SVD_MANUAL`	Walk-through script for SVD.	`SVD_MANUAL`

Helpers / demo wrappers (skip unless debugging)

Function	Role
`basicERO`, `eroPG`, `rowop`	Primitive row-operation helpers / GUI playground.
`pivot_columns`	Returns pivot column indices of an RREF matrix. Workaround for MATLAB R2025b symbolic `rref` not supporting the `[R, p] = rref(...)` two-output form. Used internally by `coordinatesInBasis`, `inSpan`, `extendToBasis`, `isLinearlyIndependent`, `polynomialInterpolation`, `rowColSpace`, `solveMatrixEquation`, `eigenAnalysis`, `generalizedEigenvector`, `changeBasisCoords`.
`print_factored_struct`	Pretty-prints the factored struct returned by `gramSchmidt`.
`runEigenAnalysis`, `run_orthonormalize`, `run_svd`, `checkspan_script`	Demos that call the underlying functions on canned examples.

---

2. Worked examples

Sample output is shown in the form a MATLAB session would print it. Numerical formatting may vary slightly across versions.

`solveLinearSystem_RREF(A,b)`

Example exam question (Tutorial 1 / AY2122 Q2(b)). *"Is the system $A\mathbf{x} = \mathbf{b}$ consistent? If yes, find the general solution."*

Solve $3x + 2y - z = 1,\ x + 2y + z = 3,\ x + z = 2$.

$$A = \begin{pmatrix} 3 & 2 & -1 \\ 1 & 2 & 1 \\ 1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 3 \\ 2 \end{pmatrix}$$

```
A = [3 2 -1; 1 2 1; 1 0 1];
b = [1; 3; 2];
solveLinearSystem_RREF(A, b);
```

```
RREF: [1 0 0 1/2; 0 1 0 1/2; 0 0 1 3/2]
Status: Unique Solution.   x = [1/2; 1/2; 3/2]
```

Symbolic by default — `A` and `b` may contain parameters (`syms a b; solveLinearSystem_RREF([1 0 3 1; 3 a 9 0; 2 0 a+6 a; 2 0 6 b], [2; 6; b+2; b+2])` works and prints the parametric general solution).

`classify_linear_system(M)`

Example exam question (AY2021 Q1, AY2122 Q4(a), AY2324 Q12). *"Find the conditions on a, b such that the system has (i) no solution, (ii) a unique solution, (iii) infinitely many solutions; in case (iii) write down a general solution."*

For the parametric augmented matrix

$$M = \begin{pmatrix} 1 & a & 2 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & a & 2 \end{pmatrix}.$$

```
syms a;
M = [1 a 2 0; 1 0 1 1; 1 0 a 2];
classify_linear_system(M);

% Equivalent — pass A and b separately:
A = [1 a 2; 1 0 1; 1 0 a];
b = [0; 1; 2];
classify_linear_system(A, b);
```

Abbreviated output:

```
Critical values for 'a': 0, 1
General case (a ≠ 0, 1):  UNIQUE SOLUTION
a = 0:  INFINITE SOLUTIONS (1 parameter(s): s)
        Free parameter(s): s   (x_3 = s)
a = 1:  NO SOLUTION
```

Infinite-solutions cases are split by the number of free parameters: 1 parameter is labelled `INFINITE SOLUTIONS (1 PARAMETER s)`, 2 parameters as `INFINITE SOLUTIONS (2 PARAMETERS s, t)`, and 3 or more as `INFINITE SOLUTIONS (3+ PARAMETERS)`. Each case also prints the literal mapping (e.g. `x_3 = s`) so the parameter-to-free-variable correspondence is unambiguous in exam working. The returned `result` struct exposes a `num_params` field alongside `classification` for programmatic use.

Infinite-solution branches additionally print the parametric general solution, a particular `x_p` (free vars = 0), and a null-space basis. Special cases are grouped by classification in the output. Critical conditions are detected from four sources: denominators of the parametric RREF, consistency rows of the form `[0 ⋯ 0 | f(a)]`, **rank-drop conditions** of the coefficient matrix (`det(A)` for square `A`; `det(AᵀA)` for tall, `det(AAᵀ)` for wide — by Cauchy–Binet these are sums of squared maximal minors and vanish exactly when the rank drops), and the **left null space** of the original `A` (catches conditions that symbolic `rref` hides when it normalises a parametric divisor). The rank-drop check matters for non-square parametric systems (e.g. AY2425 makeup midterm Q1, a 5×4 system whose unique-solution regime breaks at `a = 0` and `a = b` — invisible to the RREF denominator scan). When the primary rank polynomial is **identically zero** (e.g. a column of `A` is zero for every value of the parameters, so the rank is already deficient generically) the solver falls back to enumerating all `r × r` minors of `A` (where `r` is the generic rank); this catches the "extra" parameter case where the rank drops *further*,

which is what produces 2-parameter (rather than 1-parameter) infinite-solution branches in problems like AY2425 final Q1 (`a = 0, b = 0` 2-parameter case after `a = 0` already gives a 1-parameter family). Solutions that introduce the imaginary unit (e.g. quadratic factors with negative discriminant like `12a² + 4ab + 11b²`) are filtered out so only real critical conditions are enumerated.

Up to 4 parameters. When a substitution leaves remaining symbolic variables, the function recurses on the substituted matrix, so joint conditions like `(a = 0 AND b = 2)` are reached. Every top-level call ends with a roll-up summary table:

```
============================================================
  CLASSIFICATION SUMMARY (joint conditions, leaf cases)
============================================================
  Case   Conditions     Classification
  -----  -------------  --------------------
  (1  )  a = 0,  b ≠ 2  NO SOLUTION
  (2  )  a = 0,  b = 2  INFINITE SOLUTIONS (2 PARAMETERS)
  (3  )  a ≠ 0,  b = 2  INFINITE SOLUTIONS (1 PARAMETER)
  (4  )  a ≠ 0,  b ≠ 2  UNIQUE SOLUTION
============================================================
```

`symbolicREF(A)` and `symbolicRREF(A)`

Example exam question (Tutorial 2). *"Reduce the parametric matrix A to row-echelon (or reduced row-echelon) form, branching on the pivot parameter where necessary."*

```
syms a;
A = [1 a 2; 1 0 1; 1 0 a];
R = symbolicREF(A);   % symbolic REF, no early division by 'a'
symbolicRREF(A);      % exact symbolic RREF with case branching
```

Sample output:

$$R = \begin{pmatrix} 1 & a & 2 \\ 0 & -a & -1 \\ 0 & 0 & a-1 \end{pmatrix}.$$

`symbolicRREF` further branches on $a = 0$ and $a = 1$ explicitly.

`findEROsequence(A,B)`

Example exam question (Tutorial 2 / Midterm). *"State a sequence of elementary row operations that transforms A into B."*

```
A = [1 2; 3 4];
B = [3 4; 1 2];
findEROsequence(A, B);
```

```
%% Sequence of EROs to go from A to B:
C([1 2],:) = C([2 1],:); % swap R1 <-> R2
```

`elementaryMatrices(ops, n)`

Example exam question (Midterm). *"Express the row reduction $A \to R$ as $E_k \cdots E_2 E_1 A = R$ where each E_i is an elementary matrix; hence write $A = E_1^{-1} \cdots E_k^{-1} R$."*

Mid-term style row sequence: $R_3 + 2R_1$, $R_1 \leftrightarrow R_2$, $-2R_2$, $R_3 - R_2$ on a 3×3 matrix.

```
ops = { {'add',3,1,2}, {'swap',1,2}, {'scale',2,-2}, {'add',3,2,-1} };
[Es, E, Einv] = elementaryMatrices(ops, 3);
% Then:  R = E * A,  and  A = Einv * R.
```

`E` is the composite $E_4 E_3 E_2 E_1$; `Einv` is $E_1^{-1} E_2^{-1} E_3^{-1} E_4^{-1}$ so that $A = E^{-1} R$.

`parametricCases(A, {'a',[v1 v2]}, ...)`

Example exam question (Tutorial 2 supplementary). *"For each of the values $a = -1, 0, 1$, classify the system $A\mathbf{x} = \mathbf{0}$."*

Substitute a parameter at multiple values and print each RREF.

```
syms a;
A = [1 a 1; a 1 1];
parametricCases(A, {'a', [-1 0 1]});
```

Each case prints the substituted matrix, its RREF, and a classification (INCONSISTENT / UNIQUE / INFINITE).

`solveMatrixEquation(M, B)`

Example exam question (Midterm Q4 / Make-up Q3, AY2324 Q8/Q11/Q20). *"Given $A\mathbf{v}_i = \mathbf{w}_i$ for several pairs (or $AM = B$), recover the matrix A."*

Recover A from $AM = B$ where M is 3×5 and B is 3×5 (mid-term Q4 style).

```
M = sym([-1 1 0 1 2; 0 0 3 3 2; -1 -1 2 -1 1]);
B = sym([-3 -1 7 2 6; 1 1 1 4 1; -5 -1 15 8 13]);
solveMatrixEquation(M, B);
```

→ `A = [1 1 2; 0 1 -1; 2 3 3]`.

With known null-space data (make-up Q3 style):

```
M = sym([1 1; 0 -1; 1 0]);   B = sym([2 1; -1 3; 4 0]);
N = sym([1; -1; 2]);                      % A*N = 0
solveMatrixEquation(M, B, 'NullVec', N);
```

`cofactor_expansion(A)`

Example exam question (Tutorial 3). *"Compute $\det(A)$ by cofactor expansion along a chosen row or column. Show your workings."*

Compute $\det \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{pmatrix}$.

```
cofactor_expansion([1 2 3; 0 4 5; 0 0 6]);
```

```
Expanding along Column 1 (max zeros).   det = +1 · |4 5; 0 6| = 24.
```

`cofactorMatrix(A)`

Example exam question (Tutorial 3). *"Find the cofactor matrix of A, hence write down the adjugate and use it to compute A^{-1}."*

```
A = [1 2; 3 4];
C = cofactorMatrix(A);
```

→ `C = [4 -3; -2 1]`, so `A⁻¹ = (1/det A) · Cᵀ = (1/-2) · [4 -2; -3 1]`.

`inSpan(v, S)`

Example exam question (Tutorial 4 / AY2122 Q1©). *"Show that* $\mathbf{w}$ *does (or does not) belong to* $\mathrm{span}(S)$*."*

```
[tf, c] = inSpan(sym([2;3;1]), sym([1 0; 1 1; 0 1]));
```

prints `v IS in span(S). c = [2; 1]` (so `S*c = v`). For a vector outside the span: `inSpan(sym([1;0;1]), sym([1;1;0]))` prints `v is NOT in span(S).`

`coordinatesInBasis(v, B)`

Example exam question (AY2122 Q1(e), AY2021 Q3©). *"Find* $[\mathbf{w}]_S$*, the coordinate vector of* $\mathbf{w}$ *relative to the basis* S*."*

Find $[\mathbf{v}]_S$ where $S = \{(-1,1,0),(1,0,1)\}$ and $\mathbf{v} = (3,-1,2)$.

`coordinatesInBasis(sym([3; -1; 2]), sym([-1 1; 1 0; 0 1]))` prints `[v]_B = [-1; 2]` (so `B * [v]_B = v`).

`isLinearlyIndependent(A)`

Example exam question (Tutorial 4). *"Determine whether the set* $\{\mathbf{v}_1,\ldots,\mathbf{v}_k\}$ *is linearly independent. If dependent, give a non-trivial relation."*

```
isLinearlyIndependent(sym([1 0; 0 1]));            % independent
isLinearlyIndependent(sym([1 2; 2 4]));            % dependent
isLinearlyIndependent(sym([1 0 0; 0 0 0; 0 1 0])); % contains zero vector
```

```
[1 0; 0 1]              => INDEPENDENT (rank 2 of 2)
[1 2; 2 4]              => DEPENDENT,  relation: 2*v1 - v2 = 0
[1 0 0; 0 0 0; 0 1 0]   => DEPENDENT,  contains zero column
```

`compare_spans(A,B)`

Example exam question (Tutorial 4). *"Show that* $\mathrm{span}(S) = \mathrm{span}(T)$ *— equivalently, that* S *and* T *are bases for the same subspace."*

`compare_spans(sym([1 1; 1 2; 1 0]), sym([1 0; 1 1; 1 -1]))` prints `Rank(A) = 2, Rank(B) = 2. Relation: spans are equal.`

`extendToBasis(V)`

Example exam question (Tutorial 5). *"Extend the linearly independent set* $\{\mathbf{v}_1,\ldots,\mathbf{v}_k\} \subset \mathbb{R}^n$ *to a basis of* $\mathbb{R}^n$*."*

`extendToBasis(sym([1; 1; 0]))` extends to a basis of `R^3` (adding 2 standard basis vectors): `[1 0 0; 1 1 0; 0 0 1]`.

`polynomialInterpolation(xs, ys, deg)`

Example exam question (Midterm Q10, AY2021 Q2(a)). *"Find a polynomial $p(x)$ of degree $\leq n$ such that $p(x_i) = y_i$ for the given data points."*

Mid-term Q10: $p(x) = ax^3 + bx^2 + cx + d$ with $p(0) = 1,\ p(1) = 0,\ p(2) = 3$.

```
polynomialInterpolation([0 1 2], [1 0 3], 3);
```

```
Family of polynomials with 1 free parameter(s): t1
p(x) = t1*x^3 + (2 - 3*t1)*x^2 + (2*t1 - 3)*x + 1
```

Fix the leading coefficient $a = 0 \rightarrow$ unique quadratic:

```
polynomialInterpolation([0 1 2], [1 0 3], 2);
```

```
Unique polynomial.
p(x) = 2*x^2 - 3*x + 1
```

`vandermondeMatrix(xs, deg)`

Example exam question (Midterm Q10 setup, AY2021 Q2(a)). *"Set up the Vandermonde system that interpolates the points (x_i, y_i) with a polynomial of degree at most d."*

Builds the Vandermonde matrix V for given x-values up to max degree d. Output is $n \times (d+1)$ symbolic, with `V(i, j) = xs(i)^(d+1-j)` — column 1 is x^d, last column is 1. Same convention as `polynomialInterpolation`, so solving $V\mathbf{c} = \mathbf{y}$ gives $\mathbf{c} = [a_d; a_{d-1}; \ldots; a_0]$ for $p(x) = a_d x^d + \cdots + a_0$.

```
V = vandermondeMatrix([0 1 2], 2);          % 3x3, square
V = vandermondeMatrix([1 2 3 4], 5);        % 4x6, under-determined
syms a; vandermondeMatrix([1 a a^2], 2);    % parametric inputs OK
vandermondeMatrix([-1 0 1 2]);              % deg defaults to numel(xs) - 1
```

```
[ 0, 0, 1 ]
[ 1, 1, 1 ]
[ 4, 2, 1 ]
```

`deg` defaults to `numel(xs) - 1` (smallest deg that makes V square — same as MATLAB's built-in `vander`).

`subspaceFromEquations(C)`

Example exam question (AY2324 Q3, AY2021 Q3(a)/(b)). *"Let* $V = \{\mathbf{x} \in \mathbb{R}^4 : 5x_1 + 3x_2 - 2x_3 + 3x_4 = 0\}$*. Find a basis for* V*."*

When the subspace is defined by linear equations rather than a spanning set — call this on the constraint matrix:

```
[B, d] = subspaceFromEquations([5 3 -2 3]);
```

```
rank(C) = 1,  dim(V) = 3   (V ⊂ R^4)
Basis for V (columns, integer-scaled): [-3; 5; 0; 0],  [2; 0; 5; 0],  [-3; 0; 0; 5]
Free variables: x_2 = s, x_3 = t, x_4 = r
General solution to C*x = 0:
   x_1 = (2*t)/5 - (3*s)/5 - (3*r)/5
   x_2 = s
   x_3 = t
   x_4 = r
```

The printed general solution lists each x_i in terms of the parameters and explicitly maps each parameter to its free variable (e.g. `x_2 = s, x_3 = t, x_4 = r`). The mapping uses the *raw* `null(C)` basis so that $x_{\text{free}} = \text{parameter}$ literally; the basis returned to the caller (and shown above the mapping) is the integer-scaled version (denominators cleared) for cleaner display. Free-parameter names match `least_squares` / `solveLinearSystem_RREF`. For multiple equations stack them as rows of `C`.

`changeBasisCoords(coordsS, S, T)`

Example exam question (AY2324 Q17(b), AY2223 Q19(b)). *"Suppose* $\mathbf{w}$ *is a vector in* V *with* $[\mathbf{w}]_S = (3, 1, 2)^T$*. Find* $[\mathbf{w}]_T$*, the coordinates of* $\mathbf{w}$ *relative to the basis* T*."*

Convert $[\mathbf{w}]_S$ to $[\mathbf{w}]_T$ when S and T are two bases of the same subspace. Solves $T\,[\mathbf{w}]_T = S\,[\mathbf{w}]_S$ symbolically. Errors if T is rank-deficient or if the resulting $\mathbf{w}$ doesn't lie in $\text{span}(T)$.

```
S = sym([1 0 1; 0 1 0; 1 1 0]);
T = sym([1 1 0; 0 1 1; 1 0 1]);
changeBasisCoords([2; 3; 1], S, T);
```

```
w = S * [w]_S = [3; 3; 5]
[w]_T = [5/2; 1/2; 5/2]    (verified T * [w]_T = w)
```

`rowColSpace(A)`

Example exam question (Tutorial 6, AY2324 Q5). *"Find a basis for the row space and column space of A. State the rank."*

```
rowColSpace([1 0 2 0; 0 1 0 2; 1 1 2 2]);
```

```
rank(A) = 2   (3x4),  pivot cols [1 2]
Row(A) basis (rows):  [1 0 2 0],  [0 1 0 2]
Col(A) basis (cols):  [1 0 1]ᵀ,   [0 1 1]ᵀ
```

`find_intersection(A,B)`

Example exam question (AY2021 Q4(d), AY2324 Q17©). *"Find a basis for the intersection $\mathrm{Col}(A) \cap \mathrm{Col}(B)$ (or for $V \cap W$ given spanning sets)."*

Basis for $\mathrm{Col}(A) \cap \mathrm{Col}(B)$.

`find_intersection([1 0; 0 1; 1 1], [1 1; 1 0; 2 1])` prints `dim(intersection) = 1. Basis: [1; 1; 2]`.

`projectOnto(w, u)`

Example exam question (Tutorial 7). *"Compute the orthogonal projection of $\mathbf{w}$ onto $\mathbf{u}$."*

$\mathrm{proj}_{\mathbf{u}}(\mathbf{w}) = \dfrac{\langle \mathbf{w}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle}\,\mathbf{u}$. The script promotes inputs to symbolic and prints every intermediate quantity — both dot products, the scalar coefficient, and the resulting vector — so you can copy the working straight onto the exam script.

```
projectOnto([1;2;3], [1;1;0])
```

```
<w, u> = 3
<u, u> = ||u||^2 = 2
c = <w,u>/<u,u> = 3/2
proj_u(w) = c*u = [3/2; 3/2; 0]
```

`decomposeProjection(w,V)`

Example exam question (AY2122 Q1(d), AY2324 Q4/Q17(a)). *"Find the projection $\mathbf{w}_p$ of $\mathbf{w}$ onto the subspace V, and the orthogonal component $\mathbf{w}_n = \mathbf{w} - \mathbf{w}_p$."*

`decomposeProjection([1;2;3], [1;1;0])` returns `wp = [3/2; 3/2; 0]` (projection onto `Col(V)`) and `wn = [-1/2; 1/2; 3]` (perpendicular).

`orthogonalComplement(V)`

Example exam question (AY2122 Q1(f), AY2021 Q3(e), AY2324 Q17(d)). *"Find a basis for $V^\perp$ (and an orthonormal basis if asked)."*

Find $V^\perp$ where V is the null-space example from the slide (V spanned by 3 columns in $\mathbb{R}^4$).

`orthogonalComplement(sym([1 -2 -1; 1 0 0; 0 1 0; 0 0 1]))` prints `V is in R^4. dim(V^perp) = 1. Basis: [1; -1; 2; 1]`.

`gramSchmidt(V)` and `orthonormalize_rational(V)`

Example exam question (AY2021 Q3(d), AY2324 Q15). *"Use the Gram–Schmidt process to convert the basis S into an orthonormal basis."*

Both functions walk through the full marker-style working: at every step they print the current input vector $\mathbf{v}_k$, each dot product $\langle \mathbf{v}_k, \mathbf{u}_j \rangle$ and $\langle \mathbf{u}_j, \mathbf{u}_j \rangle$ (or $\langle \mathbf{v}_k, \mathbf{w}_j \rangle$ in the orthonormal version), the scalar coefficient, the projection vector, the residual $\mathbf{u}_k = \mathbf{v}_k - \sum_j (\text{projections})$, the norm $\|\mathbf{u}_k\|$, and the unit vector $\mathbf{w}_k = \mathbf{u}_k / \|\mathbf{u}_k\|$ — **all printed as exact fractions/surds**, not decimals.

- `gramSchmidt(V)` promotes the input to symbolic internally so every printed quantity stays as a fraction or surd. Dependence is checked first via `isAlways(||u_k|| == 0)`; if that's inconclusive, falls back to a numeric tolerance (default `1e-10`, override via `gramSchmidt(V, tol)`). After the per-vector workings it prints the orthogonal set U symbolically, the orthonormal set in factored form $(1/\sqrt{c})\,\mathbf{z}$, and the verification matrix $Q^T Q$. The returned `U` and `Q` are **numeric matrices** for downstream computation.
- `orthonormalize_rational(V)` is the **exact symbolic** version — fractions and surds throughout, returns a sym matrix `E`. Errors only if a column is linearly dependent on earlier ones (the residual collapses to zero exactly).

```
V = [1 1 0; 1 0 1; 0 1 1];
gramSchmidt(V);              % symbolic internally, factored final display, numeric returns
orthonormalize_rational(V);  % fully symbolic returns
```

```
--- Processing v2 ---
  v2 = [1, 0, 1]
  Projection onto u1:
    <v2, u1> = 1
    <u1, u1> = 2
    c1 = <v2,u1>/<u1,u1> = 1/2
    proj_{u1}(v2) = c1 * u1 = [1/2, 1/2, 0]
  u2 = v2 - (sum of projections) = [1/2, -1/2, 1]
  ||u2|| = 6^(1/2)/2
  q2 = u2 / ||u2|| = [6^(1/2)/6, -6^(1/2)/6, 6^(1/2)/3]
```

Sample orthonormal basis of `Col(V)`: `Q = { (1/√2)[1,1,0], (1/√6)[1,-1,2], (1/√3)[-1,1,1] }`.

`isOrthogonal(V)` and `isOrthonormal(V)`

Example exam question (AY2324 Q6). *"Verify that the columns of V form an orthogonal (or orthonormal) set; equivalently, decide which given vector is orthogonal to every column of V."*

`isOrthogonal([1 1; 1 -1])` → "All columns are orthogonal!". `isOrthonormal(sym([1/sqrt(sym(2)), 1/sqrt(sym(2)); 1/sqrt(sym(2)), -1/sqrt(sym(2))]))` → "All columns are orthonormal (exact).".

`least_squares(A, v)`

Example exam question (AY2122 Q2©/(d), AY2021 Q2(b)(ii), AY2324 Q7/Q18). *"Find all least-squares solutions to $A\mathbf{x} = \mathbf{v}$. Hence find the orthogonal projection $\mathbf{p} = A\hat{\mathbf{x}}$ and the least-squares error $\|\mathbf{v} - \mathbf{p}\|$."*

Solves the normal equations $A^T A\mathbf{x} = A^T\mathbf{v}$ via symbolic RREF and returns the full geometry of the projection: the least-squares solution $\hat{\mathbf{x}}$, the projection $\mathbf{p} = A\hat{\mathbf{x}}$, the orthogonal component $\mathbf{w}_o = \mathbf{v} - \mathbf{p}$, and the error norm $\|\mathbf{w}_o\|$. The script also prints the **sum of squared residuals** $\|\mathbf{w}_o\|^2 = \sum_i (v_i - p_i)^2$ — exam questions vary between asking for the distance $\|\mathbf{w}_o\|$ and the squared residual sum $\|\mathbf{w}_o\|^2$, so both are shown. Each printed result is followed by a 5-significant-figure decimal version (`vpa(..., 5)`) for quick numerical sanity checks.

```
A = [1 1; 1 2; 1 3];
v = [2; 2; 4];
[x, p, Wo, dist] = least_squares(A, v);
```

```
A'A = [3 6; 6 14],  A'v = [8; 18]
x_hat = [2/3; 1]                        (≈ 0.6667, 1.0000)
projection p   = [5/3; 8/3; 11/3]       (≈ 1.667, 2.667, 3.667)
residual v - p = [1/3; -2/3; 1/3]
distance       = sqrt(6)/3              (≈ 0.8165)
sum of squared residuals  ||v - p||^2 = 2/3   (≈ 0.6667)
```

When $\mathrm{rank}(A) < n$ the routine prints a parametric general solution `xp + s*n1 + t*n2 + …` (free parameters auto-named `s, t, r, t1, t2, …`) along with the explicit mapping (e.g. `Free variables: x_2 = s, x_4 = t`) so you can read off which variable each parameter represents. The projection $\mathbf{p}$ is still computed from the particular solution.

`charPoly(A)`

Example exam question (AY2122 Q4(a), AY2324 Q20©). *"Find the characteristic polynomial of A. State the algebraic and geometric multiplicities of each eigenvalue."*

```
charPoly([2 1 0; 0 2 0; 0 0 5]);
```

```
char(A) = (x - 2)^2 * (x - 5)
λ = 2:  am = 2,  gm = 1
λ = 5:  am = 1,  gm = 1
```

`eigenAnalysis(A)` and `orthogonalDiagonalize(A)`

Example exam question (AY2021 Q5(a), AY2122 Q3, AY2223 Q4(b)/(d), AY2324 Q13). *"Find an invertible matrix P and a diagonal matrix D such that $A = PDP^{-1}$ — or, if A is symmetric, an orthogonal P with $A = PDP^T$. If A is not diagonalizable, find the Jordan form."*

```
[P,D] = eigenAnalysis([4 1; 2 3]);        % general: eigenspaces + diagonalisation
[P,D] = eigenAnalysis(A, false);          % skip Gram-Schmidt + normalisation when A is symmetric
orthogonalDiagonalize([2 1; 1 2]);        % symmetric => orthogonal P
```

`eigenAnalysis` prints, for every eigenvalue, the algebraic and geometric multiplicities and an explicit eigenspace basis from `null(λI - A)` (matching the `det(xI - A)` convention used for the characteristic polynomial). Each basis vector is rescaled to clear rational denominators and any common integer factor — e.g. $[-1/5,\ 1/5,\ 1,\ 1]^T$ is printed as $[-1,\ 1,\ 5,\ 5]^T$. Returns `P, D` only if every $\mathrm{gm} = \mathrm{am}$; otherwise both come back empty.

Symmetric case (orthogonal diagonalisation). When `A` is symmetric and `force_orthogonal_if_symmetric` is left at its default `true`, the script walks through the marker-style pipeline within each eigenspace: prints the raw basis vectors $\mathbf{a}_1, \mathbf{a}_2, \ldots$, applies Gram–Schmidt explicitly with the projection-subtraction formula $\mathbf{v}_k = \mathbf{a}_k - \sum_{j<k} \frac{\mathbf{a}_k \cdot \mathbf{v}_j}{\mathbf{v}_j \cdot \mathbf{v}_j} \mathbf{v}_j$ (the coefficient is printed inline), then normalises each $\mathbf{v}_k$ to $\mathbf{w}_k = \mathbf{v}_k / \|\mathbf{v}_k\|$ with $\|\mathbf{v}_k\|$ shown. A 1-dimensional eigenspace skips Gram–Schmidt and just normalises. The script also reminds you once that cross-eigenspace orthogonality is automatic for symmetric `A`. Pass `false` as the second argument to suppress this and produce a non-orthogonal `P` instead.

For the general case: `P = [1 -1; 1 2]`, `D = diag(5, 2)`, `A = P D P⁻¹`.
For the symmetric case: `P = [1/√2, 1/√2; -1/√2, 1/√2]`, `D = diag(1, 3)`, verified `A = P D Pᵀ`.

Defective eigenvalues (gm < am). When some eigenvalue has fewer eigenvectors than its algebraic multiplicity, `eigenAnalysis` no longer returns empty. It builds a Jordan chain by solving $(A - \lambda I)\mathbf{v}_{k+1} = \mathbf{v}_k$ for each missing direction, prints every chain vector, assembles them next to the eigenvector(s) in `P`, and returns the **Jordan form** J in place of D (with 1's on the superdiagonal between chain members). The returned matrices satisfy $AP = PJ$ (verified automatically). The simple gm=1 chain is the common exam case; the multi-chain case (1 < gm < am) is also handled via null-space dimensions of $(A - \lambda I)^k$. Each generalized eigenvector $\mathbf{v}_k$ (for $k \geq 2$) is also printed in **general-solution form** $\mathbf{v}_k = \mathrm{particular} + s\,\mathbf{e}_1 + t\,\mathbf{e}_2 + \cdots$ where $\mathbf{e}_i$ span the λ-eigenspace (free parameters auto-named `s, t, r, t1, t2, …`). **Chain elements are not fraction-cleared** — rescaling a single $\mathbf{v}_k$ by an LCM would break $(A - \lambda I)\mathbf{v}_k = \mathbf{v}_{k-1}$, so $\mathbf{v}_k$ is left with whatever fractions the RREF / null-space step produces. The pairing then holds exactly (e.g. for $A = [3, -1; 4, -1]$, $\mathbf{v}_1 = [1; 2]^T$, $\mathbf{v}_2 = [\frac{1}{2}; 0]^T$ — not $[1; 0]^T$, which would satisfy $(A - I)\mathbf{v}_2 = 2\mathbf{v}_1$).

```
[P, J] = eigenAnalysis([-4 -4 5; 3 4 -3; 1 2 0]);   % AY2122 Q3, repeated eigenvalue λ=1
```

```
λ = 1:  am = 2, gm = 1  → defective, build Jordan chain.
   v_1 (eigenvector)              = [1; 0; 1]
   v_2 (generalized, M·v2 = v1)   = [-1; 1; 0]
   General form (s free):  v_2    = [s - 1; 1; s]
P = [-2 1 -1; 1 0 1; 0 1 0],  J = [-2 0 0; 0 1 1; 0 0 1]
Verified: A * P = P * J.
```

Complex / irrational eigenvalues. `solve` is called with `'MaxDegree', 3`, so cubics are returned in explicit Cardano form rather than as `root(z^3 - …, z, k)` placeholders. When the resulting symbolic expression is still unreadable (contains `root(` or exceeds 80 chars), the function falls back to a `vpa(·, 6)` numeric form for the eigenvalue header, the eigenspace basis, and the final `P` / `D` printout. Sorting of the diagonal is done by `(real, imag)` of the numeric value so complex conjugate pairs don't break `sort()`.

```
eigenAnalysis([1 1 0; 0 2 1; 1 0 1]);   % char poly x^3 - 4x^2 + 5x - 3 (1 real + 2 complex roots)
```

```
λ ≈ 0.560 - 0.606i  (vpa, 6 s.f.)
basis ≈ [-0.440 - 0.606i;  0.194 - 0.533i;  1]
```

`calculateSteadyState(P)`

Example exam question (AY2122 Q2(d), AY2324 Q2). *"In the long run, what is the steady-state distribution of this Markov chain?"*

```
P = sym([7 4; 3 6]) / 10;
calculateSteadyState(P);
```

→ `π = [4/7; 3/7]`, with `P · π = π`.

`isStochastic(P)` and `isRegularStochastic(P)`

Example exam question (Tutorial 9). *"Verify that P is a (regular) stochastic matrix and hence has a unique steady-state distribution."*

```
P = [0.5 0.2; 0.5 0.8];
isStochastic(P);
isRegularStochastic(P);
```

```
P is column-stochastic (2x2).   Regular: P^1 has all entries > 0.
```

`wronskian(X)`

Example exam question (Tutorial 11 Q3, AY2021 Q6(b)). *"Use the Wronskian to verify that the fundamental set of solutions you found is linearly independent."*

`wronskian([exp(t); exp(t)], [exp(2*t); 2*exp(2*t)])` prints `W(t) = exp(3*t) ≠ 0 ⇒ linearly independent.`

`scalarODEtoSystem(coeffs)`

Example exam question (AY2324 Q16(a), Tutorial 11 Q2). *"Convert the second-order ODE* $y'' - 6y' + 9y = 0$ *to a first-order homogeneous system* $\mathbf{y}' = A\mathbf{y}$*. Hence solve the IVP. (Other methods earn 0 marks.)"*

Convert a scalar n-th order ODE $c_n y^{(n)} + c_{n-1} y^{(n-1)} + \cdots + c_0 y = 0$ into the companion-matrix system $Y' = AY$ with $Y = [y;\, y';\, \ldots;\, y^{(n-1)}]$. Pass `coeffs` in **highest-order-first** order, matching how the equation is written. Required by AY2324 final Q16, where the rubric awards 0 marks for any other approach.

```
A = scalarODEtoSystem([1 -6 9]);     % y'' - 6y' + 9y = 0
solveLinearODE(A, [1; 4], 0);        % IVP y(0)=1, y'(0)=4
```

```
ODE: y'' + (-6)*y' + (9)*y = 0
Companion matrix:
[  0   1 ]
[ -9   6 ]

(then solveLinearODE recovers)
y(t)  = e^{3t}(t + 1)
y'(t) = e^{3t}(3t + 4)
```

If the leading coefficient isn't 1 the function divides through automatically and prints the normalised equation.

`solveLinearODE(A, x0, t0)`

Example exam question (AY2021 Q5(b)/Q6, AY2122 Q3(b), AY2223 Q3, AY2324 Q16(b)/Q20(d), Tutorial 11). *"Solve the system* $\mathbf{y}' = A\mathbf{y}$ *subject to the initial condition* $\mathbf{y}(0) = \mathbf{y}_0$*. Show your workings clearly."*

The function first lists the **fundamental solutions** $\mathbf{x}_1(t), \ldots, \mathbf{x}_n(t)$ (one per column of a fundamental matrix), then the general/particular solution. Each $\mathbf{x}_k(t)$ is rescaled to clear rational denominators — fundamental solutions are unique up to a nonzero scalar, so e.g. an eigenvector $[\frac{1}{2}, \frac{1}{2}, 1]^T$ from `jordan()` is reported as $[1, 1, 2]^T$ for cleaner workings.

Distinct real eigenvalues:

```
solveLinearODE([0 1; -2 -3]);
```

```
x_1(t) = [1; -1] e^{-t},   x_2(t) = [1; -2] e^{-2t}
x(t)   = c_1 [1; -1] e^{-t} + c_2 [1; -2] e^{-2t}
```

Complex eigenvalues (handled in real form — no `1i` leakage):

```
solveLinearODE([0 -1; 1 0]);
```

For $\lambda = \pm i$ with eigenvector $\mathbf{u} \pm i\mathbf{w}$, the function emits

`x₁(t) = u cos t - w sin t`, `x₂(t) = u sin t + w cos t`, then `x(t) = c₁ x₁(t) + c₂ x₂(t)`.

Trig is kept in the canonical Chapter 7 form $\mathbf{P}\cos(\beta t) - \mathbf{Q}\sin(\beta t)$. Rows like `cos(t) - sin(t)` are **not** collapsed into phase-shifted form (e.g. $\sqrt{2}\cos(t + \pi/4)$) — the function deliberately skips `simplify()` on the trig output to match the slides verbatim.

Repeated, deficient eigenvalue (Jordan chain $\mathbf{v}_1, \mathbf{v}_2$):

```
solveLinearODE([3 1; 0 3]);
```

`x₁(t) = v₁ e^{3t}`, `x₂(t) = (t v₁ + v₂) e^{3t}`, `x(t) = e^{3t} [c₁ + c₂ t; c₂]`.

For a length-k Jordan chain the j-th solution is $\left(\sum_{i=0}^{j-1} \frac{t^i}{i!}\mathbf{v}_{j-i}\right) e^{\lambda t}$.

IVP:

```
solveLinearODE([0 1; -2 -3], [1; 0], 0);
```

→ `x(t) = [2 e^{-t} - e^{-2t}; -2 e^{-t} + 2 e^{-2t}]`.

The script also prints the recovered constants $c_1, \ldots, c_n$ obtained from $\Phi(t_0)\,\mathbf{c} = \mathbf{x}_0$.

`generalizedEigenvector(A, lambda, v1)`

Example exam question (AY2122 Q3(a)(ii), AY2223 Q4©, AY2324 Q20(a)). *"Find a generalized eigenvector* $\mathbf{v}_2$ *associated to the repeated eigenvalue* λ*, satisfying* $(A - \lambda I)\mathbf{v}_2 = \mathbf{v}_1$*."* (Note: `eigenAnalysis` now builds the full chain automatically — use this only for one-off chain steps.)

For $A = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$, eigenvalue $\lambda = 3$, eigenvector $\mathbf{v}_1 = (1, 0)^T$:

`generalizedEigenvector([3 1; 0 3], 3, [1;0])` returns `v2 = [0; 1]` (free vars = 0; verified `(A - λI) v2 = v1`).

`svd_exact(A)`

Example exam question (beyond core syllabus). *"Find a singular value decomposition* $A = USV^T$ *of the given matrix."* Useful when SVD appears as a bonus or in past papers from chapters past the standard final.

```
[U,S,V] = svd_exact([1 0 1; -1 -2 0; 0 1 -1]);
```

Returns the symbolic singular value decomposition $A = USV^T$. Useful when SVD appears beyond the main syllabus or in past papers.

---

3. Naming aliases / footguns

- `eigenAnalysis.m` is the workhorse: eigenspaces + diagonalisation, and Jordan chains when defective; `orthogonalDiagonalize.m` specialises to the symmetric case. Use `generalizedEigenvector(A, λ, v1)` only when you want a single chain step in isolation — `eigenAnalysis` already builds the full chain end-to-end.
- `decomposeProjection.m` returns both $\mathbf{w}_p$ and $\mathbf{w}_n$ — prefer it over a manual `projectOnto` chain when you need the perpendicular component.
- `inSpan` answers "is this vector in the span?"; `coordinatesInBasis` answers "what are its coordinates?" (and errors if the input isn't a basis or v isn't in span).
- `isOrthogonal` uses a decimal tolerance; `isOrthonormal` uses exact symbolic equality. For exam workings, prefer the symbolic one.
- Demo scripts (`runEigenAnalysis`, `run_orthonormalize`, `run_svd`, `checkspan_script`) just call the underlying functions on a canned example — not for direct use in problem sets.